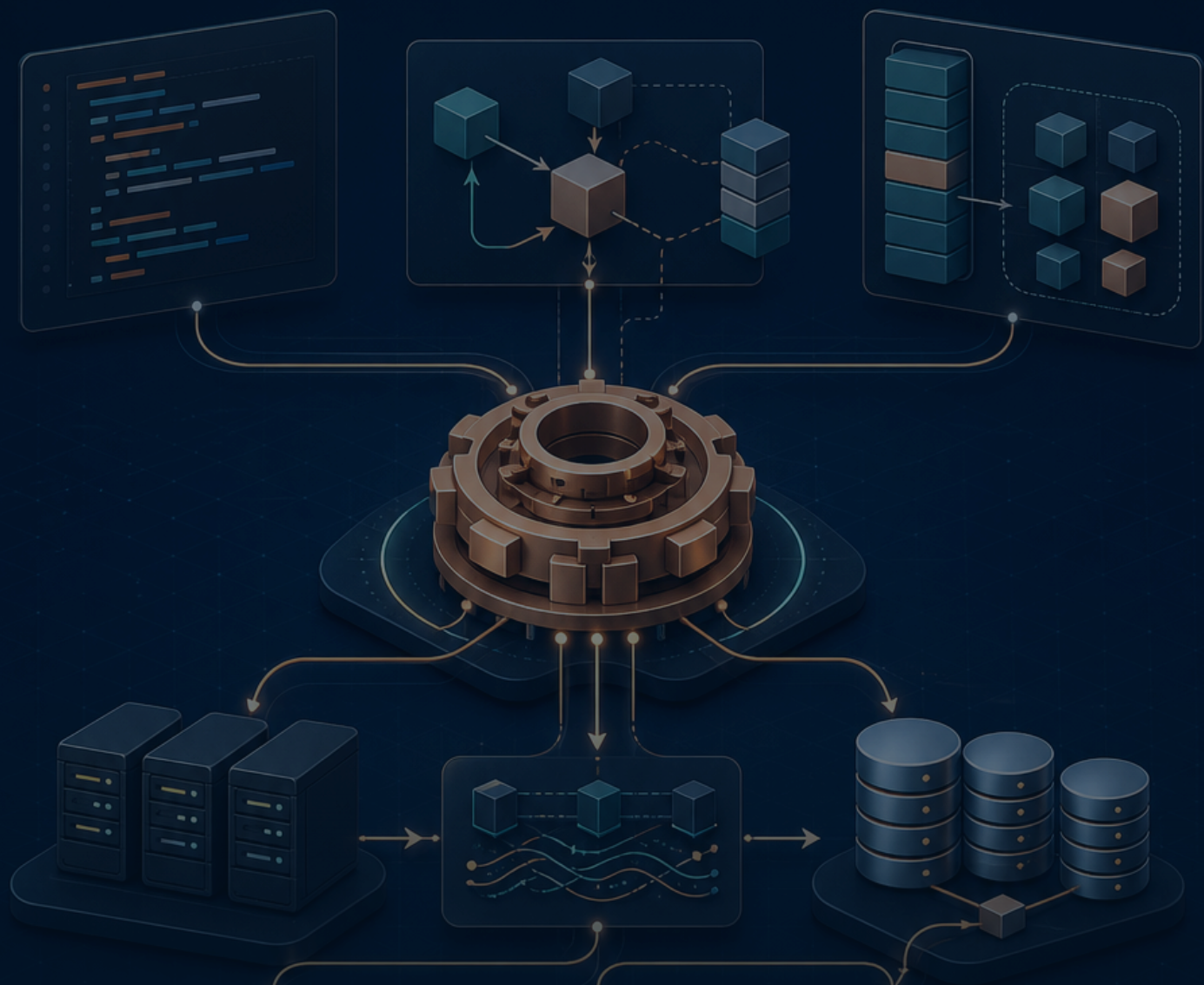




VOLUME 01 / RUST BACKEND MASTERY

環境構築 — Windows・WSL・Rust・ Topcoat・Turso・Fly.io

初回セットアップから、各ツールの役割、確認コマンド、つまずきの切り分けまでを積み上げる

10

深掘り単元

20+

実行・解答例

40+

図・表・画面

実務

設計と障害対応

LEARNING MAP

この巻で身に付けること

インストール手順の羅列ではなく、PATH、シェル、CPU、ツールチェーン、プロジェクト、クラウド認証の依存関係を理解します。

到達目標

- ✓ PowerShellとWSLのどちらで実行中か判別できる
- ✓ rustup、rustc、cargo、Topcoat CLIを区別できる
- ✓ TursoとFly.ioへ安全に疎通確認できる
- ✓ PATH・ポート・リンカ問題を切り分けられる

学習地図

01 作業フォルダとPowerShell

環境構築の最初に、作業ルート、現在位置、シェル、OS、CPUを記録します。短い英数字パスを使うと、初学時に空白・権限・文字コードによる差を減らせます。

02 Git・改行コード・秘密の除外

Gitは変更履歴を小さな単位で保存します。WindowsのCRLFとLinuxのLF、秘密ファイル、生成物を最初に決めると、DockerやCIだけで壊れる問題を減らせます。

03 VS Codeとrust-analyzer

VS Codeは編集と統合端末、rust-analyzerは型情報による補完・診断・定義移動を担当します。エディタの赤線と端末のcargo checkが同じプロジェクトを見ているか確認します。

04 rustup・rustc・Cargo

rustupはツールチェーン管理、rustcはコンパイラ、Cargoはプロジェクト・依存・ビルド・テスト管理です。三つを混同せず、PATHからどの実行ファイルが呼ばれたか確認します。

05 MSVC Build ToolsとCPUアーキテクチャ

WindowsのMSVCターゲットはリンク時にMicrosoft C++ Build Toolsを使います。x64とARM64、MSVCとGNUの不一致は、ネイティブ依存をビルドするときに表示化します。

06 Cargo品質コマンド

checkは高速な型検査、fmtは整形、clippyはLint、testはテスト、docはAPI文書です。目的を分け、保存時・commit前・CIで同じ順序を使います。

07 WSL2とDocker

WSL2はWindows上のLinux開発環境、Dockerはアプリと依存をイメージとして再現する仕組みです。Windows側とWSL側で別々のrustupやPATHを持つことを理解します。

08 Topcoat 0.5.0とCLI

Topcoatは公式がearly-stage / experimentalと明記しています。教材ではcrateとCLIを0.5.0へ完全固定し、mainブランチの新しいAPIを混ぜません。

09 Turso CLIとlibSQL疎通

Turso CLIはDB作成・トークン発行・SQL実行に使い、Rustアプリはlibsql crateで接続します。Windowsでは公式CLIをWSLから使う前提を明確にします。

10 flyctlと環境診断

flyctlはFly.io上のアプリ作成、秘密設定、デプロイ、ログ、SSHに使います。最初からdeployせず、認証、組織、アプリ名、リージョン、Dockerビルドを順に確認します。

HOW TO USE THIS VOLUME

読むだけで終わらせない進め方

「予想→実行→観察→一か所変更→回帰テスト」を一単位ごとに繰り返します。写経した行数ではなく、入力・処理・出力・失敗を自分の言葉で説明できるかを進捗にします。

1. 予想

実行前に入力型、出力、失敗箇所を予想します。

2. そのまま実行

最初は変更せず、教材と同じ出力が比較します。

3. 一か所変更

値または型を一つだけ変え、差分の理由を説明します。

4. 回帰テスト

直した失敗を自動テストへ残し、再発を防ぎます。

STEP 1

作業ルートの決定

STEP 2

Get-Location

STEP 3

PowerShell版

STEP 4

OSとCPUの確認

毎回そろえる確認コマンド

● ● ● 単元を終える前の品質確認

```
cargo fmt --check
cargo clippy --all-targets -- -D warnings
cargo test --locked --all-targets
```

対象	この教材の基準	混在を防ぐ理由
Rust	2024 Edition / rust-version 1.95以上	editionとMSRVを明示し、環境差を再現可能にする
Topcoat	crate / CLI ともに 0.5.0固定	early-stageのため、mainブランチの別構文を混ぜない
DB・認証	Turso/libSQL、Argon2id、Topcoat Session	一つのRustプロセスで責務とデータの流れを追える
公開	Docker + Fly.io、HOST=0.0.0.0、PORT=8080	ローカルとコンテナの待受差を明示する

重要な版の前提 Topcoatは公式がearly-stage / experimentalと明記しています。本教材はcrateとCLIを **0.5.0** に固定し、mainブランチの別構文を混ぜません。

UNIT 01

作業フォルダとPowerShell

UNIT 01・CONCEPT

作業フォルダとPowerShell

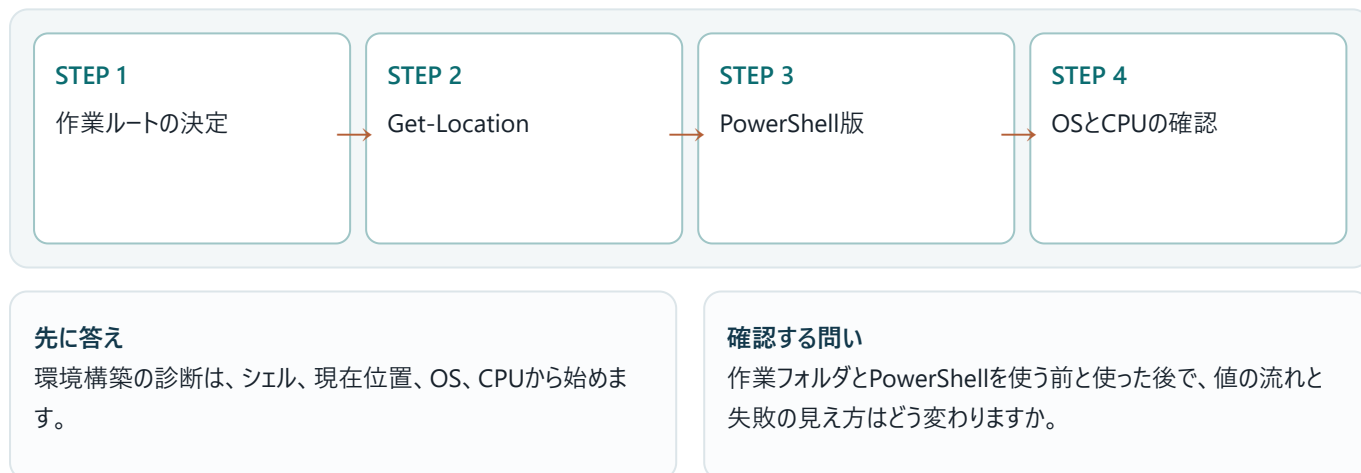
環境構築の最初に、作業ルート、現在位置、シェル、OS、CPUを記録します。短い英数字パスを使うと、初学時に空白・権限・文字コードによる差を減らせます。

この単元の到達点

- 作業ルートの決定
- Get-Location
- PowerShell版
- OSとCPUの確認

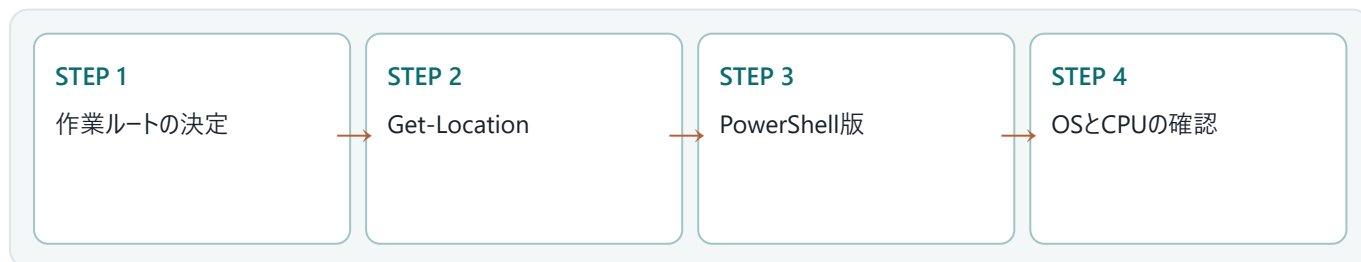
なぜバックエンドで必要か

開発、テスト、Dockerで同じプロジェクトルートを使い、手順には現在位置を明記して再現性を上げます。



UNIT 01・MENTAL MODEL

作業フォルダとPowerShellを図でつかむ



順序	観察対象	初心者が確認すること
1	作業ルートの決定	作業ルートの決定 — 入力と前提を確認し、境界を曖昧にしない。
2	Get-Location	Get-Location — 値がどの順で変換されるかを追跡する。
3	PowerShell版	PowerShell版 — 失敗を再現し、型またはログから原因を特定する。
4	OSとCPUの確認	OSとCPUの確認 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

作業フォルダとPowerShellとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 01 · MINIMUM CODE

まず動く最小コード

作業フォルダとPowerShellだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
$root='C:\dev\rust-learning'  
New-Item -ItemType Directory -Force -Path $root | Out-Null  
Set-Location $root  
Get-Location  
$PSVersionTable.PSVersion
```

● ● ● 実行コマンド

通常権限のPowerShellで一行ずつ実行

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力

ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力

標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 01 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>\$root='C:\dev\rust-learning'</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>New-Item -ItemType Directory -Force -Path \$root Out-Null</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>Set-Location \$root</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>Get-Location</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>\$PSVersionTable.PSVersion</code>	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力が作業ルートの決定の条件を満たすか確認する。
- ✓ Get-Locationを実行し、途中の型と状態を追う。
- ✓ PowerShell版で失敗を成功経路から分離する。
- ✓ OSとCPUの確認により、出力と副作用を検証する。

型を声に出す 作業フォルダとPowerShellに関する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 01・CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

```

● ● ● 実行結果

Path
----
C:\dev\rust-learning

Major Minor Patch
7      5      2

```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 01・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
$root='C:\dev\rust-learning\diagnosis'
New-Item -ItemType Directory -Force -Path $root | Out-Null
Set-Location $root
Get-Location
$PSVersionTable.PSVersion
Get-ComputerInfo | Select-Object WindowsProductName,OsArchitecture
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

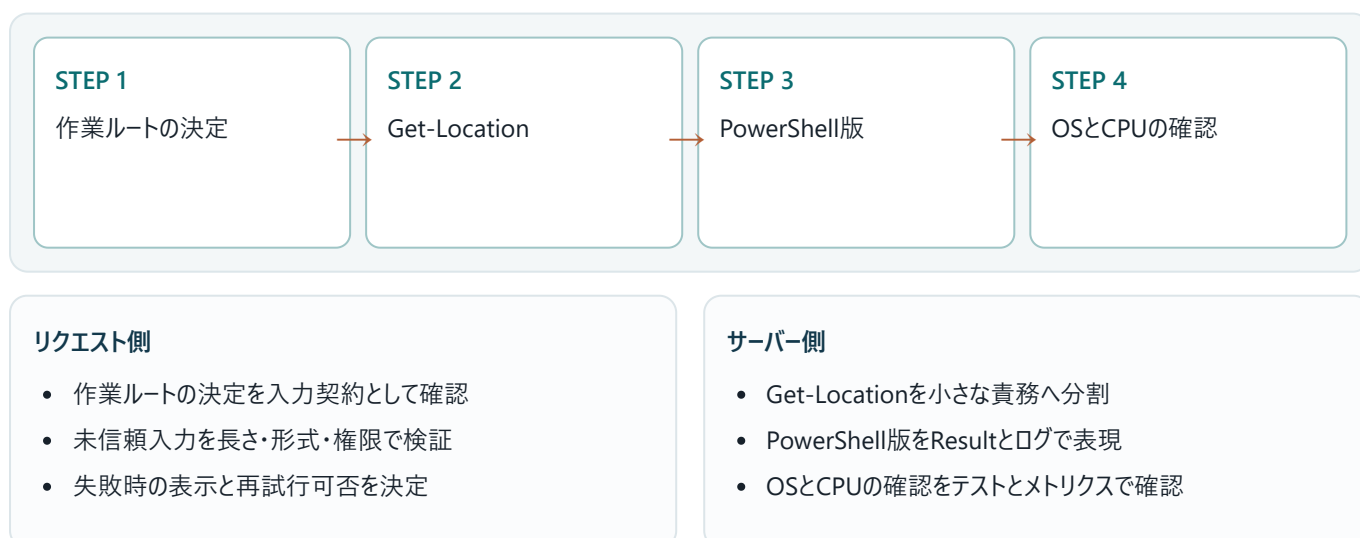
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 01・BACKEND APPLICATION

バックエンドのどこで使うか

開発、テスト、Dockerで同じプロジェクトルートを使い、手順には現在位置を明記して再現性を上げます。



境界で守る不変条件

- ✓ 不正な入力副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 01・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	作業ルートの決定に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	Get-Locationの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	PowerShell版またはOSとCPUの確認の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 01・DEBUGGING

作業フォルダとPowerShellを再現して切り分ける

作業フォルダとPowerShellの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: 作業ルートの決定
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: Get-Location
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: PowerShell版
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: OSとCPUの確認

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	作業ルートの決定	作業ルートの決定 — 入力と前提を確認し、境界を曖昧にしない。
2	Get-Location	Get-Location — 値がどの順で変換されるかを追跡する。
3	PowerShell版	PowerShell版 — 失敗を再現し、型またはログから原因を特定する。
4	OSとCPUの確認	OSとCPUの確認 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 環境構築の診断は、シェル、現在位置、OS、CPUから始めます。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 01 ・ PRACTICE

手を動かす演習

diagnosisフォルダを作り、現在位置、PowerShell版、OS名、CPUアーキテクチャを表示してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 01・SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
$root='C:\dev\rust-learning\diagnosis'  
New-Item -ItemType Directory -Force -Path $root | Out-Null  
Set-Location $root  
Get-Location  
$PSVersionTable.PSVersion  
Get-ComputerInfo | Select-Object WindowsProductName,OsArchitecture
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 環境構築の診断は、シェル、現在位置、OS、CPUから始めます。
- ✓ 作業フォルダとPowerShellとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 02

Git・改行コード・秘密の除外

UNIT 02 ・ CONCEPT

Git・改行コード・秘密の除外

Gitは変更履歴を小さな単位で保存します。WindowsのCRLFとLinuxのLF、秘密ファイル、生成物を最初に決めると、DockerやCIだけで壊れる問題を減らせます。

この単元の到達点

- git initとstatus
- .gitignore
- LFとCRLF
- Cargo.lockの追跡

なぜバックエンドで必要か

Cargo.lock、migrations、fly.tomlは履歴へ入れ、target、.env、秘密鍵は除外します。アプリはCargo.lockを固定して再現性を優先します。



UNIT 02 ・ MENTAL MODEL

Git・改行コード・秘密の除外を図でつかむ



Gitは変更履歴を小さな単位で保存します。WindowsのCRLFとLinuxのLF、秘密ファイル、生成物を最初に決めると、DockerやCIだけで壊れる問題を減らせます。図では左から右へ処理を追います。各箱の入口では入力の種類と信頼度、出口では結果の種類と副作用を確認しま

す。

順序	観察対象	初心者が確認すること
1	git initとstatus	git initとstatus — 入力と前提を確認し、境界を曖昧にしない。
2	.gitignore	.gitignore — 値がどの順で変換されるかを追跡する。
3	LFとCRLF	LFとCRLF — 失敗を再現し、型またはログから原因を特定する。
4	Cargo.lockの追跡	Cargo.lockの追跡 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

Git・改行コード・秘密の除外とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 02・MINIMUM CODE

まず動く最小コード

Git・改行コード・秘密の除外だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
git --version
git init
git status
git config --get core.autocrlf
```

●●● 実行コマンド

Git for Windows導入後にPowerShellで実行

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力

ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力

標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 02・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>git --version</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>git init</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>git status</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>git config --get core.autocrlf</code>	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力値が`git init`と`status`の条件を満たすか確認する。
- ✓ `.gitignore`を実行し、途中の型と状態を追う。
- ✓ LFとCRLFで失敗を成功経路から分離する。
- ✓ `Cargo.lock`の追跡により、出力と副作用を検証する。

型を声に出す Git・改行コード・秘密の除外に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値の一つを変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 02 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

```
git version 2.x
Initialized empty Git repository
On branch main
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力値が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 02 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
# .gitignore
/target
.env
.env.*
!.env.example

# commands
git add .gitignore Cargo.lock
git status
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 02 · BACKEND APPLICATION

バックエンドのどこで使うか

Cargo.lock、migrations、fly.tomlは履歴へ入れ、target、.env、秘密鍵は除外します。アプリはCargo.lockを固定して再現性を優先します。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 02 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	git initとstatusに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	.gitignoreの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	LFとCRLFまたはCargo.lockの追跡の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 02・DEBUGGING

Git・改行コード・秘密の除外を再現して切り分ける

Git・改行コード・秘密の除外の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: git initとstatus
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: .gitignore
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: LFとCRLF
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: Cargo.lockの追跡

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	git initとstatus	git initとstatus — 入力と前提を確認し、境界を曖昧にしない。
2	.gitignore	.gitignore — 値がどの順で変換されるかを追跡する。
3	LFとCRLF	LFとCRLF — 失敗を再現し、型またはログから原因を特定する。
4	Cargo.lockの追跡	Cargo.lockの追跡 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 生成物と秘密は除外し、依存固定とDB移行は履歴へ入れます。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 02 ・ PRACTICE

手を動かす演習

.gitignoreへtargetと.envを追加し、Cargo.lockは追跡されることをgit statusで確認してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 02 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
# .gitignore
/target
.env
.env.*
!.env.example

# commands
git add .gitignore Cargo.lock
git status
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 生成物と秘密は除外し、依存固定とDB移行は履歴へ入れます。
- ✓ Git・改行コード・秘密の除外とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 03

VS Codeとrust-analyzer

UNIT 03 · CONCEPT

VS Codeとrust-analyzer

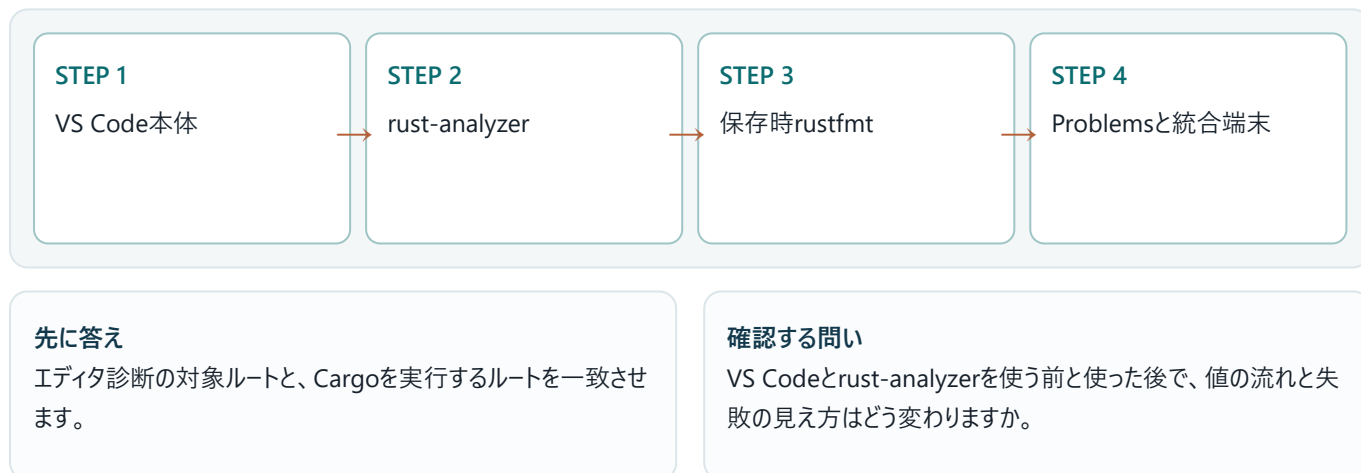
VS Codeは編集と統合端末、rust-analyzerは型情報による補完・診断・定義移動を担当します。エディタの赤線と端末のcargo checkが同じプロジェクトを見ているか確認します。

この単元の到達点

- VS Code本体
- rust-analyzer
- 保存時rustfmt
- Problemsと統合端末

なぜバックエンドで必要か

エディタ診断は補助です。commit前とCIではプロジェクトルートから同じCargoコマンドを実行します。



UNIT 03 · MENTAL MODEL

VS Codeとrust-analyzerを図でつかむ



VS Codeは編集と統合端末、rust-analyzerは型情報による補完・診断・定義移動を担当します。エディタの赤線と端末のcargo checkが同じプロジェクトを見ているか確認します。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	VS Code本体	VS Code本体 — 入力と前提を確認し、境界を曖昧にしない。
2	rust-analyzer	rust-analyzer — 値がどの順で変換されるかを追跡する。
3	保存時rustfmt	保存時rustfmt — 失敗を再現し、型またはログから原因を特定する。
4	Problemsと統合端末	Problemsと統合端末 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

VS Codeとrust-analyzerとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 03 · MINIMUM CODE

まず動く最小コード

VS Codeとrust-analyzerだけに注目した最小例です。まず変更せず実行し、出力が一致した後に一行ずつ意味を確認します。

```
{
  "editor.formatOnSave": true,
  "rust-analyzer.check.command": "clippy",
  "files.eol": "\n"
}
```

●●● 実行コマンド

プロジェクトルートで `code .` を実行

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 03 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
{	この行が受け取る値、作る値、副作用の有無を確認します。
"editor.formatOnSave": true,	この行が受け取る値、作る値、副作用の有無を確認します。
"rust-analyzer.check.command": "clippy",	この行が受け取る値、作る値、副作用の有無を確認します。
"files.eol": "\n"	この行が受け取る値、作る値、副作用の有無を確認します。
}	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力がVS Code本体の条件を満たすか確認する。
- ✓ rust-analyzerを実行し、途中の型と状態を追う。
- ✓ 保存時rustfmtで失敗を成功経路から分離する。
- ✓ Problemsと統合端末により、出力と副作用を検証する。

型を声に出す VS Codeとrust-analyzerに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 03 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● http://localhost:3000/learn

Rust Backend Lab

実行する

VS Codeとrust-analyzer

左にファイル、中央にRustコード、下にcargo checkの結果が表示されます。赤線は保存前に型エラーを知らせます。

観察ポイント

- VS Code本体
- rust-analyzer
- 保存時rustfmt

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 03・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
fn main() {
    let message = "formatted by rustfmt";
    println!("{}", message);
}

// settings.jsonではformatOnSaveをtrueにする
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

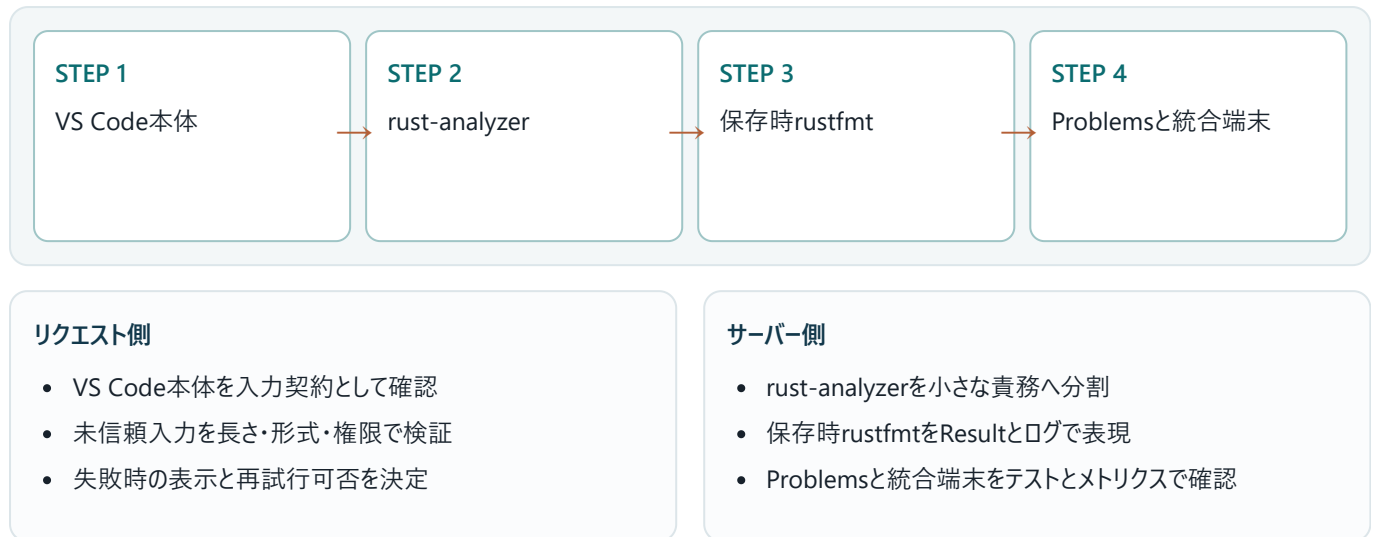
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 03・BACKEND APPLICATION

バックエンドのどこで使うか

エディタ診断は補助です。commit前とCIではプロジェクトルートから同じCargoコマンドを実行します。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 03 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	VS Code本体に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	rust-analyzerの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	保存時rustfmtまたはProblemsと統合端末の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 03 · DEBUGGING

VS Codeとrust-analyzerを再現して切り分ける

VS Codeとrust-analyzerの問題を、環境・入力・型・副作用の順に切り分けます。

1. STEP 1 — エラー全文と再現コマンドを保存する。対象: VS Code本体
2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: rust-analyzer
3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: 保存時rustfmt
4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: Problemsと統合端末

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	VS Code本体	VS Code本体 — 入力と前提を確認し、境界を曖昧にしない。
2	rust-analyzer	rust-analyzer — 値がどの順で変換されるかを追跡する。
3	保存時rustfmt	保存時rustfmt — 失敗を再現し、型またはログから原因を特定する。
4	Problemsと統合端末	Problemsと統合端末 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 エディタ診断の対象ルートと、Cargoを実行するルートを一致させます。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 03 ・ PRACTICE

手を動かす演習

未使用変数と崩したインデントを作り、保存前後とcargo clippyの結果を比較してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 03 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
fn main() {  
    let message = "formatted by rustfmt";  
    println!("{}", message);  
}  
  
// settings.jsonではformatOnSaveをtrueにする
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ エディタ診断の対象ルートと、Cargoを実行するルートを一致させます。
- ✓ VS Codeとrust-analyzerとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 04

rustup・rustc・Cargo

UNIT 04・CONCEPT

rustup・rustc・Cargo

rustupはツールチェーン管理、rustcはコンパイラ、Cargoはプロジェクト・依存・ビルド・テスト管理です。三つを混同せず、PATHからどの実行ファイルが呼ばれたか確認します。

この単元の到達点

- rustup toolchain
- rustcコンパイル
- Cargoプロジェクト
- PATHと実行元

なぜバックエンドで必要か

CIとDockerでもrust-versionとeditionを合わせます。rustup updateだけでは依存クレートの変化を固定できないためCargo.lockも必要です。



先に答え

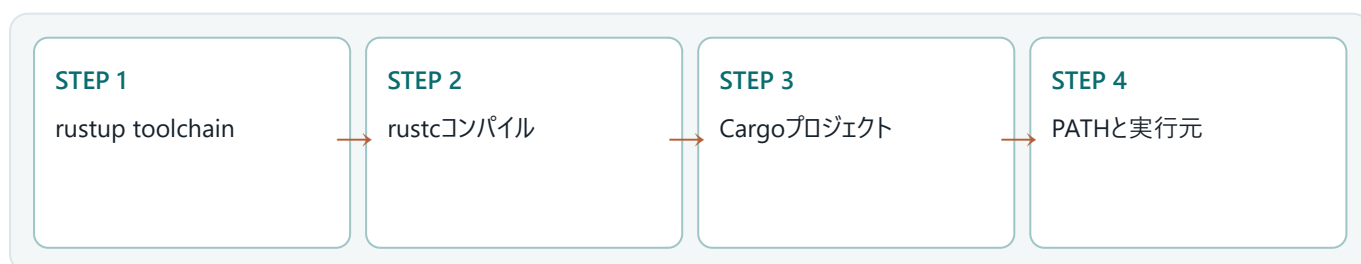
rustupは工具箱、rustcは翻訳機、Cargoはプロジェクト監督です。

確認する問い

rustup・rustc・Cargoを使う前と使った後で、値の流れと失敗の見える方はどう変わりますか。

UNIT 04・MENTAL MODEL

rustup・rustc・Cargoを図でつかむ



rustupはツールチェーン管理、rustcはコンパイラ、Cargoはプロジェクト・依存・ビルド・テスト管理です。三つを混同せず、PATHからどの実行ファイルが呼ばれたか確認します。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認

します。

順序	観察対象	初心者が確認すること
1	rustup toolchain	rustup toolchain — 入力と前提を確認し、境界を曖昧にしない。
2	rustcコンパイル	rustcコンパイル — 値がどの順で変換されるかを追跡する。
3	Cargoプロジェクト	Cargoプロジェクト — 失敗を再現し、型またはログから原因を特定する。
4	PATHと実行元	PATHと実行元 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

rustup・rustc・Cargoとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 04 · MINIMUM CODE

まず動く最小コード

rustup・rustc・Cargoだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
rustup show
rustc --version --verbose
cargo --version
Get-Command rustc,cargo,rustup | Select-Object Name,Source
```

●●● 実行コマンド

winget install Rustlang.Rustup の後、新しいPowerShellで実行

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力

ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力

標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 04 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>rustup show</code>	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。
<code>rustc --version --verbose</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>cargo --version</code>	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。
Get-Command rustc,cargo,rustup Select-Object Name,Source	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力 rustup toolchain の条件を満たすか確認する。
- ✓ rustc コンパイルを実行し、途中の型と状態を追う。
- ✓ Cargo プロジェクトで失敗を成功経路から分離する。
- ✓ PATH と実行元により、出力と副作用を検証する。

型を声に出す rustup・rustc・Cargo に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 04 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

```
stable-x86_64-pc-windows-msvc (default)
rustc 1.x.x
cargo 1.x.x
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTP ステータス、標準エラー、ログ、DB の行数など、画面に現れない観測点も合わせて確認します。

UNIT 04 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
# rust-toolchain.toml
[toolchain]
channel = "stable"
components = ["rustfmt", "clippy"]

# commands
cargo new hello-rust
cd hello-rust
cargo run
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 04 · BACKEND APPLICATION

バックエンドのどこで使うか

CIとDockerでもrust-versionとeditionを合わせます。rustup updateだけでは依存クレートの変化を固定できないためCargo.lockも必要です。



リクエスト側

- rustup toolchainを入力契約として確認
- 未信頼入力を長さ・形式・権限で検証
- 失敗時の表示と再試行可否を決定

サーバー側

- rustcコンパイルを小さな責務へ分割
- CargoプロジェクトをResultとログで表現
- PATHと実行元をテストとメトリクスで確認

境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 04 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	rustup toolchainに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	rustcコンパイルの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	CargoプロジェクトまたはPATHと実行元の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。

✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 04・DEBUGGING

rustup・rustc・Cargoを再現して切り分ける

rustup・rustc・Cargoの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: rustup toolchain
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: rustcコンパイル
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: Cargoプロジェクト
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: PATHと実行元

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	rustup toolchain	rustup toolchain — 入力と前提を確認し、境界を曖昧にしない。
2	rustcコンパイル	rustcコンパイル — 値がどの順で変換されるかを追跡する。
3	Cargoプロジェクト	Cargoプロジェクト — 失敗を再現し、型またはログから原因を特定する。
4	PATHと実行元	PATHと実行元 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 rustupは工具箱、rustcは翻訳機、Cargoはプロジェクト監督です。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 04 ・ PRACTICE

手を動かす演習

`cargo new hello-rust`を作り、`rust-toolchain.toml`で`stable`と`rustfmt`、`clippy`を指定してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時に`panic`せず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 04 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
# rust-toolchain.toml
[toolchain]
channel = "stable"
components = ["rustfmt", "clippy"]

# commands
cargo new hello-rust
cd hello-rust
cargo run
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ rustupは工具箱、rustcは翻訳機、Cargoはプロジェクト監督です。
- ✓ rustup・rustc・Cargoとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 05

MSVC Build ToolsとCPUアーキテクチャ

UNIT 05 · CONCEPT

MSVC Build ToolsとCPUアーキテクチャ

WindowsのMSVCターゲットはリンク時にMicrosoft C++ Build Toolsを使います。x64とARM64、MSVCとGNUの不一致は、ネイティブ依存をビルドするときに表面化します。

この単元の到達点

- host triple
- MSVC linker
- x64とARM64
- ネイティブ依存

なぜバックエンドで必要か

libSQLのようにネイティブコードを含む依存はリンカ、CMake、bindgenの影響を受けます。Windowsで詰まる場合はWSL2またはDockerで切り分けます。



先に答え

リンカエラーではRustコードより先にhost triple、Build Tools、CPU、ネイティブ依存を確認します。

確認する問い

MSVC Build ToolsとCPUアーキテクチャを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。

UNIT 05 · MENTAL MODEL

MSVC Build ToolsとCPUアーキテクチャを図でつかむ



WindowsのMSVCターゲットはリンク時にMicrosoft C++ Build Toolsを使います。x64とARM64、MSVCとGNUの不一致は、ネイティブ依存をビルドするときに表面化します。図では左から右へ処理を追います。各箱の入口では入力 の型と信頼度、出口では結果の型と副作用を

確認します。

順序	観察対象	初心者が確認すること
1	host triple	host triple — 入力と前提を確認し、境界を曖昧にしない。
2	MSVC linker	MSVC linker — 値がどの順で変換されるかを追跡する。
3	x64とARM64	x64とARM64 — 失敗を再現し、型またはログから原因を特定する。
4	ネイティブ依存	ネイティブ依存 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

MSVC Build ToolsとCPUアーキテクチャとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 05 · MINIMUM CODE

まず動く最小コード

MSVC Build ToolsとCPUアーキテクチャだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
rustc -vV
rustup target list --installed
Get-CimInstance Win32_OperatingSystem | Select-Object OSArchitecture
```

● ● ● 実行コマンド

Developer PowerShell for VSで cargo build --release

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 05 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>rustc -vV</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>rustup target list --installed</code>	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。
<code>Get-CimInstance Win32_OperatingSystem Select-Object OSArchitecture</code>	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力がある triple の条件を満たすか確認する。
- ✓ MSVC linker を実行し、途中の型と状態を追う。
- ✓ x64 と ARM64 で失敗を成功経路から分離する。
- ✓ ネイティブ依存により、出力と副作用を検証する。

型を声に出す MSVC Build Tools と CPU アーキテクチャに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 05 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

●●● 実行結果

```
host: x86_64-pc-windows-msvc
x86_64-pc-windows-msvc
OSArchitecture: 64-bit
```

見える結果
端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと
型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTP ステータス、標準エラー、ログ、DB の行数など、画面に現れない観測点も合わせて確認します。

UNIT 05 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
fn main() {
    println!("arch={} os={}", std::env::consts::ARCH, std::env::consts::OS);
}

// cargo build --release
// .\target\release\hello-rust.exe
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 05 · BACKEND APPLICATION

バックエンドのどこで使うか

libSQLのようにネイティブコードを含む依存はリンカ、CMake、bindgenの影響を受けます。Windowsで詰まる場合はWSL2またはDockerで切り分けます。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。

- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 05 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	host tripleに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	MSVC linkerの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	x64とARM64またはネイティブ依存の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

error: example failed
help: 最初の原因行と型の差を確認してください

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 05・DEBUGGING

MSVC Build ToolsとCPUアーキテクチャを再現して切り分ける

MSVC Build ToolsとCPUアーキテクチャの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: host triple
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: MSVC linker
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: x64とARM64
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: ネイティブ依存

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	host triple	host triple — 入力と前提を確認し、境界を曖昧にしない。
2	MSVC linker	MSVC linker — 値がどの順で変換されるかを追跡する。
3	x64とARM64	x64とARM64 — 失敗を再現し、型またはログから原因を特定する。
4	ネイティブ依存	ネイティブ依存 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 リンカエラーではRustコードより先にhost triple、Build Tools、CPU、ネイティブ依存を確認します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 05 ・ PRACTICE

手を動かす演習

hello-rustをrelease buildし、生成された実行ファイルからOSとARCHを表示してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 05 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
fn main() {  
    println!("arch={} os={}", std::env::consts::ARCH, std::env::consts::OS);  
}  
  
// cargo build --release  
// .\target\release\hello-rust.exe
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ リンカエラーではRustコードより先にhost triple、Build Tools、CPU、ネイティブ依存を確認します。
- ✓ MSVC Build ToolsとCPUアーキテクチャとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 06

Cargo品質コマンド

UNIT 06 ・ CONCEPT

Cargo品質コマンド

checkは高速な型検査、fmtは整形、clippyはLint、testはテスト、docはAPI文書です。目的を分け、保存時・commit前・CIで同じ順序を使います。

この単元の到達点

- cargo check
- cargo fmt
- cargo clippy
- cargo testとdoc

なぜバックエンドで必要か

バックエンドではbinだけでなくtests、examples、feature組合せも壊れます。CIでは警告をエラー扱いし、ローカルと同じコマンドを使います。



UNIT 06 ・ MENTAL MODEL

Cargo品質コマンドを図でつかむ



checkは高速な型検査、fmtは整形、clippyはLint、testはテスト、docはAPI文書です。目的を分け、保存時・commit前・CIで同じ順序を使います。図では左から右へ処理を追います。各箱の入口では入力型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	cargo check	cargo check — 入力と前提を確認し、境界を曖昧にしない。
2	cargo fmt	cargo fmt — 値がどの順で変換されるかを追跡する。
3	cargo clippy	cargo clippy — 失敗を再現し、型またはログから原因を特定する。
4	cargo testとdoc	cargo testとdoc — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

Cargo品質コマンドとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 06・MINIMUM CODE まず動く最小コード

Cargo品質コマンドだけに注目した最小例です。まず変更せず実行し、出力が一致した後に一行ずつ意味を確認します。

```
cargo check --all-targets
cargo fmt --all -- --check
cargo clippy --all-targets --all-features -- -D warnings
cargo test --all-features
cargo doc --no-deps
```

●●● 実行コマンド

プロジェクトルートで上から順に実行

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力

ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力

標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 06・LINE BY LINE コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>cargo check --all-targets</code>	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。
<code>cargo fmt --all -- --check</code>	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。
<code>cargo clippy --all-targets --all-features -- -D warnings</code>	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。
<code>cargo test --all-features</code>	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。
<code>cargo doc --no-deps</code>	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。

実行時に起きる順序

- ✓ 入力がcargo checkの条件を満たすか確認する。
- ✓ cargo fmtを実行し、途中の型と状態を追う。
- ✓ cargo clippyで失敗を成功経路から分離する。
- ✓ cargo testとdocにより、出力と副作用を検証する。

型を声に出す Cargo品質コマンドに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 06 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

```
Finished `dev` profile
0 warnings
test result: ok
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 06 ・ VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
fn double(values: &[i32]) -> Vec<i32> {
    values.iter().map(|v| v * 2).collect()
}
#[test]
fn doubles_each_value() {
    assert_eq!(double(&[1, 2]), vec![2, 4]);
}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

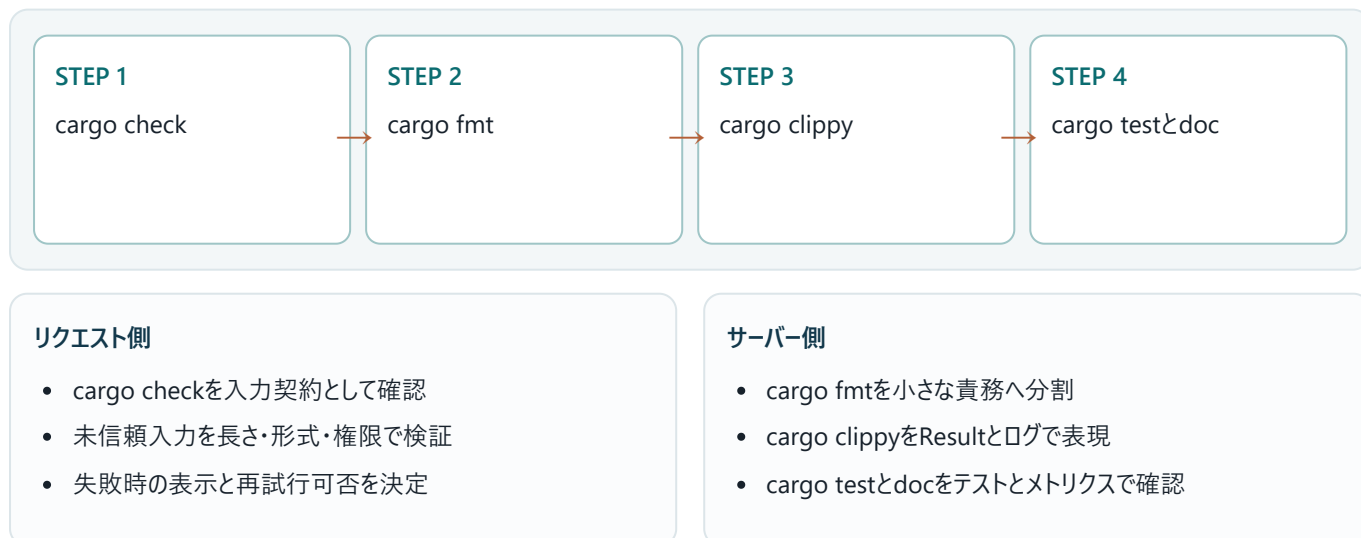
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 06 · BACKEND APPLICATION

バックエンドのどこで使うか

バックエンドではbinだけでなくtests、examples、feature組合せも壊れます。CIでは警告をエラー扱いし、ローカルと同じコマンドを使います。



境界で守る不変条件

- ✓ 不正な入力副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 06 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	cargo checkに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	cargo fmtの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	cargo clippyまたはcargo testとdocの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 06・DEBUGGING

Cargo品質コマンドを再現して切り分ける

Cargo品質コマンドの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: cargo check
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: cargo fmt
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: cargo clippy
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: cargo testとdoc

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	cargo check	cargo check — 入力と前提を確認し、境界を曖昧にしない。
2	cargo fmt	cargo fmt — 値がどの順で変換されるかを追跡する。
3	cargo clippy	cargo clippy — 失敗を再現し、型またはログから原因を特定する。
4	cargo testとdoc	cargo testとdoc — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 check・fmt・clippy・testは役割が違い、全部通って初めて変更を統合します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 06 ・ PRACTICE

手を動かす演習

Clippy警告を一つ作り、-D warningsで失敗することを確認してから修正してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 06 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
fn double(values: &[i32]) -> Vec<i32> {
    values.iter().map(|v| v * 2).collect()
}
#[test]
fn doubles_each_value() {
    assert_eq!(double(&[1, 2]), vec![2, 4]);
}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ check・fmt・clippy・testは役割が違い、全部通って初めて変更を統合します。
- ✓ Cargo品質コマンドとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 07

WSL2とDocker

UNIT 07 · CONCEPT

WSL2とDocker

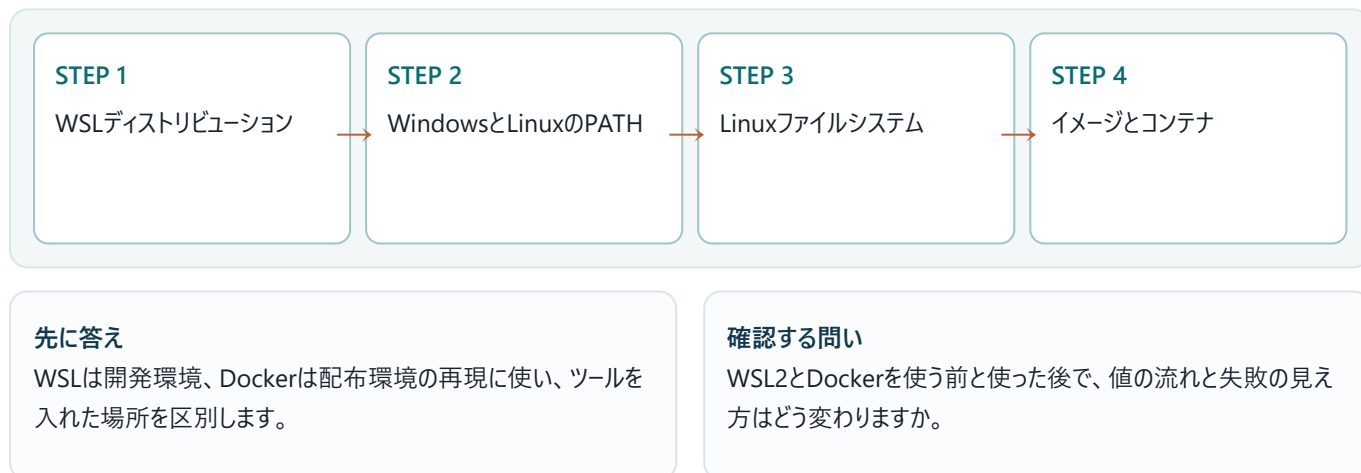
WSL2はWindows上のLinux開発環境、Dockerはアプリと依存をイメージとして再現する仕組みです。Windows側とWSL側で別々のrustupやPATHを持つことを理解します。

この単元の到達点

- WSLディストリビューション
- WindowsとLinuxのPATH
- Linuxファイルシステム
- イメージとコンテナ

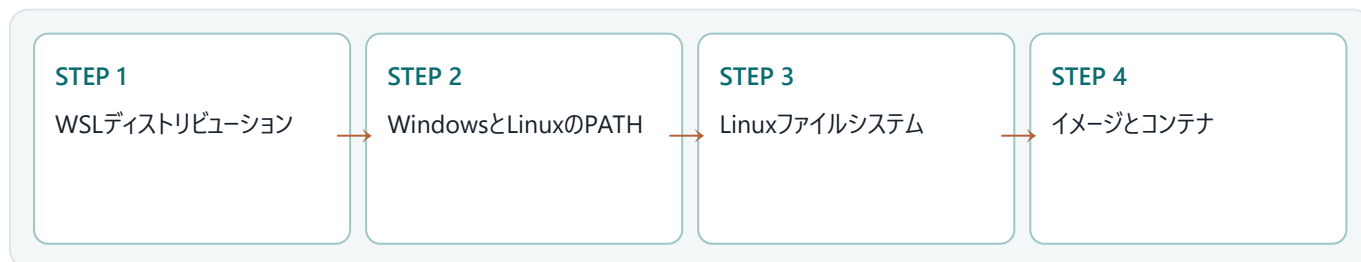
なぜバックエンドで必要か

Fly.ioはLinuxコンテナで動くため、最終ビルドをDockerで再現できるとWindows固有のリンカ差を本番へ持ち込みにくくなります。



UNIT 07 · MENTAL MODEL

WSL2とDockerを図でつかむ



WSL2はWindows上のLinux開発環境、Dockerはアプリと依存をイメージとして再現する仕組みです。Windows側とWSL側で別々のrustupやPATHを持つことを理解します。図では左から右へ処理を追います。各箱の入口では入力の方と信頼度、出口では結果の方と副作用を確認します。

順序	観察対象	初心者が確認すること
1	WSLディストリビューション	WSLディストリビューション — 入力と前提を確認し、境界を曖昧にしない。
2	WindowsとLinuxのPATH	WindowsとLinuxのPATH — 値がどの順で変換されるかを追跡する。
3	Linuxファイルシステム	Linuxファイルシステム — 失敗を再現し、型またはログから原因を特定する。
4	イメージとコンテナ	イメージとコンテナ — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

WSL2とDockerとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 07 · MINIMUM CODE

まず動く最小コード

WSL2とDockerだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
wsl --status
wsl --list --verbose
wsl -d Ubuntu -- uname -a
docker version
```

● ● ● 実行コマンド

```
docker build -t hello-rust .
docker run --rm hello-rust
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 07 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>wsl --status</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>wsl --list --verbose</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>wsl -d Ubuntu -- uname -a</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>docker version</code>	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。

実行時に起きる順序

- ✓ 入力値がWSLディストリビューションの条件を満たすか確認する。
- ✓ WindowsとLinuxのPATHを実行し、途中の型と状態を追う。
- ✓ Linuxファイルシステムで失敗を成功経路から分離する。
- ✓ イメージとコンテナにより、出力と副作用を検証する。

型を声に出す WSL2とDockerに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値の一つを変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 07 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

```
Default Distribution: Ubuntu
VERSION 2
Linux ...
Client: Docker Engine
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力値が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 07 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
# Dockerfile
FROM rust:1-bookworm AS build
WORKDIR /app
COPY . .
RUN cargo build --locked --release
FROM debian:bookworm-slim
COPY --from=build /app/target/release/hello-rust /app/hello-rust
CMD ["/app/hello-rust"]
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 07 · BACKEND APPLICATION

バックエンドのどこで使うか

Fly.ioはLinuxコンテナで動くため、最終ビルドをDockerで再現できるとWindows固有のリンカ差を本番へ持ち込みにくくなります。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 07 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	WSLディストリビューションに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	WindowsとLinuxのPATHの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	Linuxファイルシステムまたはイメージとコンテナの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 07・DEBUGGING

WSL2とDockerを再現して切り分ける

WSL2とDockerの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: WSLディストリビューション
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: WindowsとLinuxのPATH
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: Linuxファイルシステム
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: イメージとコンテナ

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	WSLディストリビューション	WSLディストリビューション — 入力と前提を確認し、境界を曖昧にしない。
2	WindowsとLinuxのPATH	WindowsとLinuxのPATH — 値がどの順で変換されるかを追跡する。
3	Linuxファイルシステム	Linuxファイルシステム — 失敗を再現し、型またはログから原因を特定する。
4	イメージとコンテナ	イメージとコンテナ — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 WSLは開発環境、Dockerは配布環境の再現に使い、ツールを入れた場所を区別します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 07 · PRACTICE

手を動かす演習

同じhello-rustをWindows、WSL、Dockerで実行し、OSとARCHの差を表にしてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 07 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
# Dockerfile
FROM rust:1-bookworm AS build
WORKDIR /app
COPY . .
RUN cargo build --locked --release
FROM debian:bookworm-slim
COPY --from=build /app/target/release/hello-rust /app/hello-rust
CMD ["/app/hello-rust"]
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ WSLは開発環境、Dockerは配布環境の再現に使い、ツールを入れた場所を区別します。
- ✓ WSL2とDockerとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 08

Topcoat 0.5.0とCLI

UNIT 08 · CONCEPT

Topcoat 0.5.0とCLI

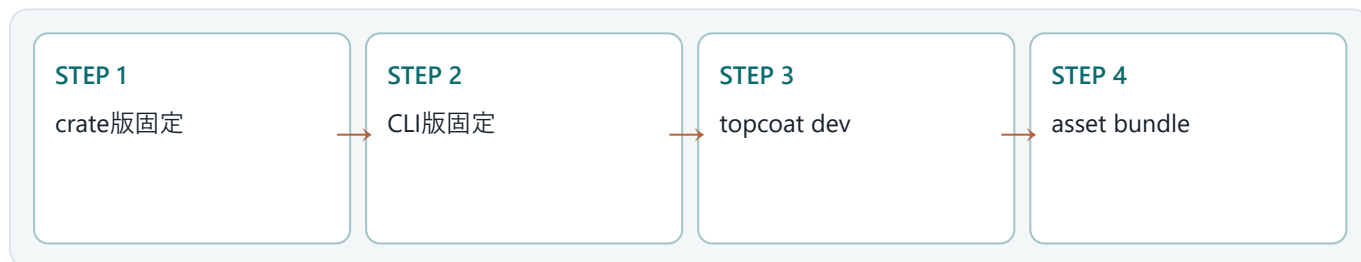
Topcoatは公式がearly-stage / experimentalと明記しています。教材ではcrateとCLIを0.5.0へ完全固定し、mainブランチの新しいAPIを混ぜません。

この単元の到達点

- crate版固定
- CLI版固定
- topcoat dev
- asset bundle

なぜバックエンドで必要か

0.5.0のAssetBundleは開発時にtopcoat dev、release時にcargo build --release後のtopcoat asset bundle --releaseが必要です。



先に答え

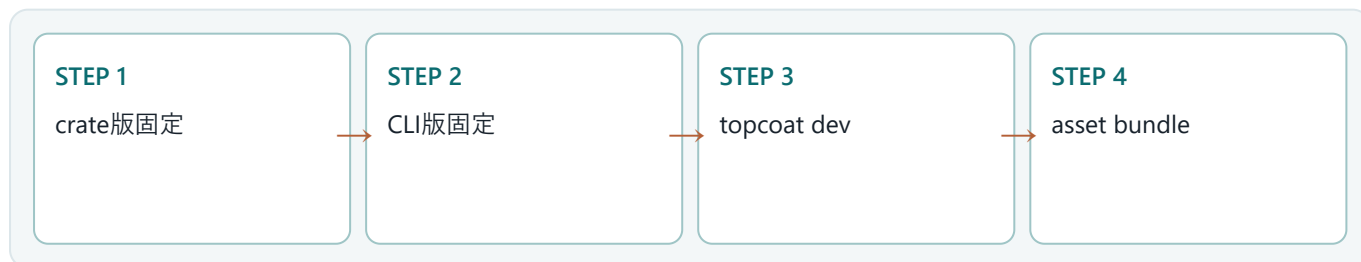
変化の速いTopcoatはcrate、CLI、Cargo.lock、参照APIを同じ版へ固定します。

確認する問い

Topcoat 0.5.0とCLIを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。

UNIT 08 · MENTAL MODEL

Topcoat 0.5.0とCLIを図でつかむ



Topcoatは公式がearly-stage / experimentalと明記しています。教材ではcrateとCLIを0.5.0へ完全固定し、mainブランチの新しいAPIを混ぜません。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	crate版固定	crate版固定 — 入力と前提を確認し、境界を曖昧にしない。
2	CLI版固定	CLI版固定 — 値がどの順で変換されるかを追跡する。
3	topcoat dev	topcoat dev — 失敗を再現し、型またはログから原因を特定する。
4	asset bundle	asset bundle — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

Topcoat 0.5.0とCLIとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 08 · MINIMUM CODE

まず動く最小コード

Topcoat 0.5.0とCLIだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
cargo install topcoat-cli --version 0.5.0 --locked
topcoat --version
# Cargo.toml: topcoat = "=0.5.0"
topcoat dev
```

● ● ● 実行コマンド

```
cargo install topcoat-cli --version 0.5.0 --locked
topcoat dev
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 08 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>cargo install topcoat-cli --version 0.5.0 --locked</code>	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。
<code>topcoat --version</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code># Cargo.toml: topcoat = "=0.5.0"</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>topcoat dev</code>	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力 crate 版固定の条件を満たすか確認する。
- ✓ CLI 版固定を実行し、途中の型と状態を追う。
- ✓ `topcoat dev` で失敗を成功経路から分離する。
- ✓ `asset bundle` により、出力と副作用を検証する。

型を声に出す Topcoat 0.5.0 と CLI に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値の一つを変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 08 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● http://localhost:3000/learn

Rust Backend Lab

実行する

Topcoat 0.5.0 と CLI

大きな見出しで Hello Topcoat と表示され、ソース保存後の再読み込みも確認できます。

観察ポイント

- crate 版固定
- CLI 版固定
- `topcoat dev`

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 08 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
use topcoat::{Result, router::{Router, page}, view::view};
#[page("/")] async fn home() -> Result { view! { <h1>"Hello Topcoat"</h1> } }
#[tokio::main] async fn main() {
    topcoat::start(Router::builder().route(home).build()).await.unwrap();
}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

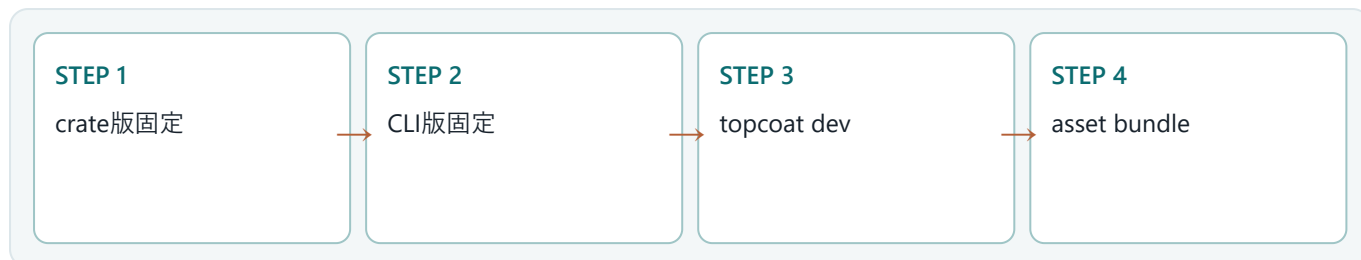
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 08 · BACKEND APPLICATION

バックエンドのどこで使うか

0.5.0のAssetBundleは開発時にtopcoat dev、release時にcargo build --release後のtopcoat asset bundle --releaseが必要です。



リクエスト側

- crate版固定を入力契約として確認
- 未信頼入力を長さ・形式・権限で検証
- 失敗時の表示と再試行可否を決定

サーバー側

- CLI版固定を小さな責務へ分割
- topcoat devをResultとログで表現
- asset bundleをテストとメトリクスで確認

境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 08・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	crate版固定に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	CLI版固定の入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	topcoat devまたはasset bundleの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 08 ・ DEBUGGING

Topcoat 0.5.0とCLIを再現して切り分ける

Topcoat 0.5.0とCLIの問題を、環境・入力・型・副作用の順に切り分けます。

1. STEP 1 — エラー全文と再現コマンドを保存する。対象: crate版固定
2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: CLI版固定
3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: topcoat dev
4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: asset bundle

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	crate版固定	crate版固定 — 入力と前提を確認し、境界を曖昧にしない。
2	CLI版固定	CLI版固定 — 値がどの順で変換されるかを追跡する。
3	topcoat dev	topcoat dev — 失敗を再現し、型またはログから原因を特定する。
4	asset bundle	asset bundle — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 変化の速いTopcoatはcrate、CLI、Cargo.lock、参照APIを同じ版へ固定します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 08 ・ PRACTICE

手を動かす演習

CLIとcrateの版を表示し、最小ページをtopcoat devで起動してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 08 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
use topcoat::{Result, router::{Router, page}, view::view};
#[page("/")] async fn home() -> Result { view! { <h1>"Hello Topcoat"</h1> } }
#[tokio::main] async fn main() {
    topcoat::start(Router::builder().route(home).build()).await.unwrap();
}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 変化の速いTopcoatはcrate、CLI、Cargo.lock、参照APIを同じ版へ固定します。
- ✓ Topcoat 0.5.0とCLIとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 09

Turso CLIとlibSQL疎通

UNIT 09 · CONCEPT

Turso CLIとlibSQL疎通

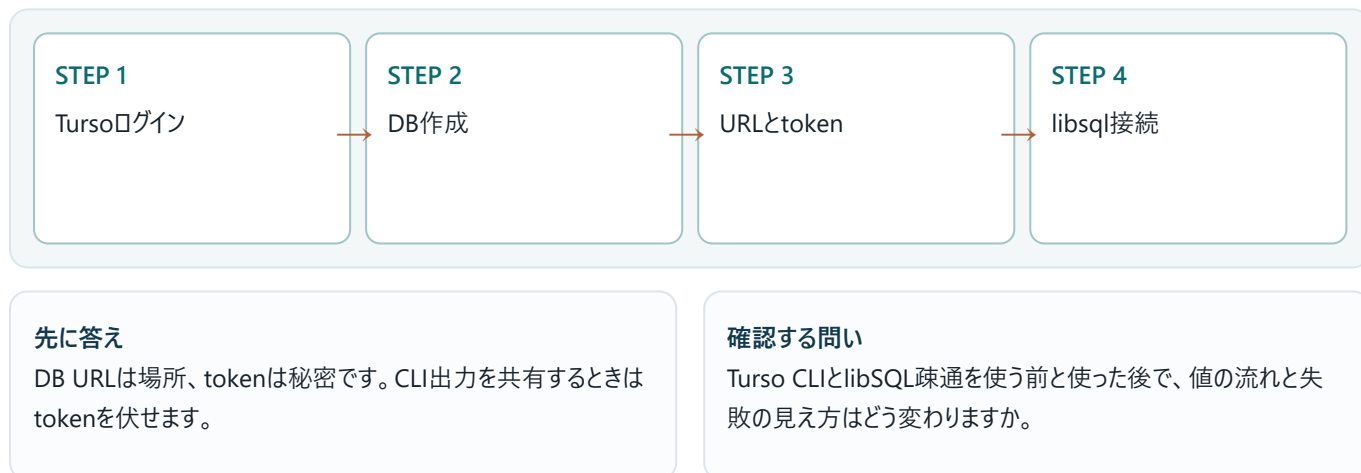
Turso CLIはDB作成・トークン発行・SQL実行に使い、Rustアプリはlibsql crateで接続します。Windowsでは公式CLIをWSLから使う前提を明確にします。

この単元の到達点

- Tursoログイン
- DB作成
- URLとtoken
- libsql接続

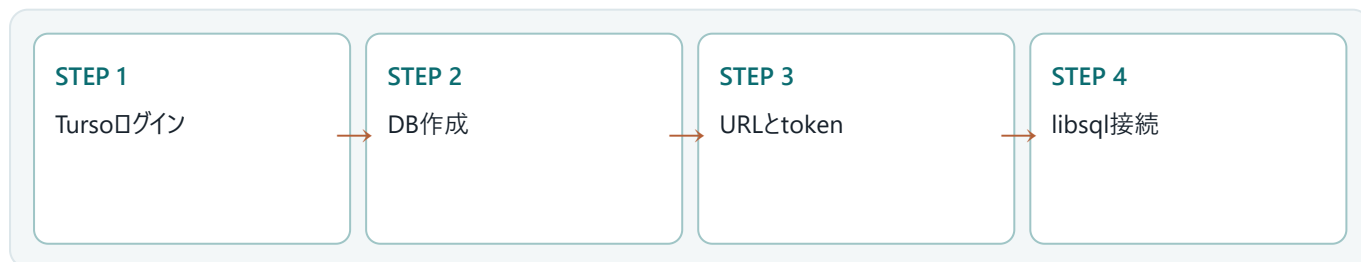
なぜバックエンドで必要か

URLは接続先、tokenは認証情報です。tokenを教材の画面やGitへ貼らず、開発では.env、本番ではfly secretsへ保存します。



UNIT 09 · MENTAL MODEL

Turso CLIとlibSQL疎通を図でつかむ



Turso CLIはDB作成・トークン発行・SQL実行に使い、Rustアプリはlibsql crateで接続します。Windowsでは公式CLIをWSLから使う前提を明確にします。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	Tursoログイン	Tursoログイン — 入力と前提を確認し、境界を曖昧にしない。
2	DB作成	DB作成 — 値がどの順で変換されるかを追跡する。
3	URLとtoken	URLとtoken — 失敗を再現し、型またはログから原因を特定する。
4	libsql接続	libsql接続 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

Turso CLIとlibSQL疎通とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 09・MINIMUM CODE

まず動く最小コード

Turso CLIとlibSQL疎通だけに注目した最小例です。まず変更せず実行し、出力が一致した後に一行ずつ意味を確認します。

```
turso auth login
turso db create rustnote-dev
turso db show rustnote-dev --url
turso db tokens create rustnote-dev
```

●●● 実行コマンド

WSL内で `turso db shell rustnote-dev`

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 09・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>turso auth login</code>	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。
<code>turso db create rustnote-dev</code>	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。
<code>turso db show rustnote-dev --url</code>	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。
<code>turso db tokens create rustnote-dev</code>	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。

実行時に起きる順序

- ✓ 入力値がTursoログインの条件を満たすか確認する。
- ✓ DB作成を実行し、途中の型と状態を追う。
- ✓ URLとtokenで失敗を成功経路から分離する。
- ✓ libsql接続により、出力と副作用を検証する。

型を声に出す Turso CLIとlibSQL疎通に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値の一つを変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 09 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

```
libsql://rustnote-dev-...turso.io
eyJ...token...
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力値が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 09 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
CREATE TABLE health_check (
  id INTEGER PRIMARY KEY,
  checked_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
);
INSERT INTO health_check DEFAULT VALUES;
SELECT * FROM health_check;
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 09 · BACKEND APPLICATION

バックエンドのどこで使うか

URLは接続先、tokenは認証情報です。tokenを教材の画面やGitへ貼らず、開発では.env、本番ではfly secretsへ保存します。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。

- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 09 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	Tursoログインに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	DB作成の入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	URLとtokenまたはlibsql接続の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

error: example failed
help: 最初の原因行と型の差を確認してください

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 09・DEBUGGING

Turso CLIとlibSQL疎通を再現して切り分ける

Turso CLIとlibSQL疎通の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: Tursoログイン
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: DB作成
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: URLとtoken
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: libsql接続

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	Tursoログイン	Tursoログイン — 入力と前提を確認し、境界を曖昧にしない。
2	DB作成	DB作成 — 値がどの順で変換されるかを追跡する。
3	URLとtoken	URLとtoken — 失敗を再現し、型またはログから原因を特定する。
4	libsql接続	libsql接続 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 DB URLは場所、tokenは秘密です。CLI出力を共有するときはtokenを伏せます。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 09 ・ PRACTICE

手を動かす演習

Turso DBへhealth_check表を作り、一行INSERTしてSELECT結果を確認してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 09 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
CREATE TABLE health_check (  
  id INTEGER PRIMARY KEY,  
  checked_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP  
);  
INSERT INTO health_check DEFAULT VALUES;  
SELECT * FROM health_check;
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ DB URLは場所、tokenは秘密です。CLI出力を共有するときはtokenを伏せます。
- ✓ Turso CLIとlibSQL疎通とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 10

flyctlと環境診断

UNIT 10 · CONCEPT

flyctlと環境診断

flyctlはFly.io上のアプリ作成、秘密設定、デプロイ、ログ SSHに使います。最初からdeployせず、認証、組織、アプリ名、リージョン、Dockerビルドを順に確認します。

この単元の到達点

- fly auth
- organization
- app nameとregion
- logsとstatus

なぜバックエンドで必要か

fly launchでfly.tomlを作り、HOST=0.0.0.0、internal_port=8080、health checkを一致させます。秘密はfly secrets setで注入します。



UNIT 10 · MENTAL MODEL

flyctlと環境診断を図でつかむ



flyctlはFly.io上のアプリ作成、秘密設定、デプロイ、ログ SSHに使います。最初からdeployせず、認証、組織、アプリ名、リージョン、Dockerビルドを順に確認します。図では左から右へ処理を追います。各箱の入口では入力型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	fly auth	fly auth — 入力と前提を確認し、境界を曖昧にしない。
2	organization	organization — 値がどの順で変換されるかを追跡する。
3	app nameとregion	app nameとregion — 失敗を再現し、型またはログから原因を特定する。
4	logsとstatus	logsとstatus — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

flyctlと環境診断とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 10 · MINIMUM CODE

まず動く最小コード

flyctlと環境診断だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
fly version
fly auth login
fly auth whoami
fly orgs list
fly platform regions
```

● ● ● 実行コマンド

```
fly launch --no-deploy
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 10 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
fly version	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。
fly auth login	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。
fly auth whoami	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。
fly orgs list	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。
fly platform regions	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。

実行時に起きる順序

- ✓ 入力 fly auth の条件を満たすか確認する。
- ✓ organization を実行し、途中の型と状態を追う。
- ✓ app name と region で失敗を成功経路から分離する。
- ✓ logs と status により、出力と副作用を検証する。

型を声に出す flyctl と環境診断に関する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値の一つを変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 10 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

```
flyctl v0.x.x
you@example.com
NAME SLUG TYPE
Tokyo nrt
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 10 ・ VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
app = "rustnote-example"
primary_region = "nrt"

[env]
HOST = "0.0.0.0"
PORT = "8080"

[http_service]
internal_port = 8080
force_https = true
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 10 · BACKEND APPLICATION

バックエンドのどこで使うか

fly launchでfly.tomlを作り、HOST=0.0.0.0、internal_port=8080、health checkを一致させます。秘密はfly secrets setで注入します。



リクエスト側

- fly authを入力契約として確認
- 未信頼入力を長さ・形式・権限で検証
- 失敗時の表示と再試行可否を決定

サーバー側

- organizationを小さな責務へ分割
- app nameとregionをResultとログで表現
- logsとstatusをテストとメトリクスで確認

境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 10 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	fly authに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	organizationの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	app nameとregionまたはlogsとstatusの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。

✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 10・DEBUGGING

flyctlと環境診断を再現して切り分ける

flyctlと環境診断の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: fly auth
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: organization
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: app nameとregion
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: logsとstatus

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	fly auth	fly auth — 入力と前提を確認し、境界を曖昧にしない。
2	organization	organization — 値がどの順で変換されるかを追跡する。
3	app nameとregion	app nameとregion — 失敗を再現し、型またはログから原因を特定する。
4	logsとstatus	logsとstatus — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 Fly.ioではアプリ、組織、リージョン、待受、秘密、ヘルスチェックを別々に確認します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 10 ・ PRACTICE

手を動かす演習

`fly launch --no-deploy`で設定だけ生成し、`internal_port`と`primary_region`の意味を説明してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 10 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
app = "rustnote-example"
primary_region = "nrt"
```

```
[env]
HOST = "0.0.0.0"
PORT = "8080"
```

```
[http_service]
internal_port = 8080
force_https = true
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ Fly.ioではアプリ、組織、リージョン、待受、秘密、ヘルスチェックを別々に確認します。
- ✓ flyctlと環境診断とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

CAPSTONE

空のWindows環境からRustNoteのhelloページを起動し、環境診断レポートを作る

10単元を一つの成果物へ統合します。動く結果だけでなく、途中のエラー、設計判断、確認コマンドも提出物です。

順	使う単元	統合する判断
01	作業フォルダとPowerShell	環境構築の診断は、シェル、現在位置、OS、CPUから始めます。
02	Git・改行コード・秘密の除外	生成物と秘密は除外し、依存固定とDB移行は履歴へ入れます。
03	VS Codeとrust-analyzer	エディタ診断の対象ルートと、Cargoを実行するルートを一致させます。
04	rustup・rustc・Cargo	rustupは工具箱、rustcは翻訳機、Cargoはプロジェクト監督です。
05	MSVC Build ToolsとCPUアーキテクチャ	リンカエラーではRustコードより先にhost triple、Build Tools、CPU、ネイティブ依存を確認します。
06	Cargo品質コマンド	check・fmt・clippy・testは役割が違い、全部通って初めて変更を統合します。
07	WSL2とDocker	WSLは開発環境、Dockerは配布環境の再現に使い、ツールを入れた場所を区別します。
08	Topcoat 0.5.0とCLI	変化の速いTopcoatはcrate、CLI、Cargo.lock、参照APIを同じ版へ固定します。
09	Turso CLIとlibSQL疎通	DB URLは場所、tokenは秘密です。CLI出力を共有するときはtokenを伏せます。
10	flyctlと環境診断	Fly.ioではアプリ、組織、リージョン、待受、秘密、ヘルスチェックを別々に確認します。

完了条件

- ✓ すべてのバージョンと実行パスを記録した
- ✓ fmt・clippy・testが成功した
- ✓ Topcoat crateとCLIが0.5.0で一致した
- ✓ TursoとFly.ioの秘密をGitへ含めていない

REVIEW

巻末確認問題

用語だけでなく、「小さな例」「バックエンドでの場所」「失敗時の挙動」を含めて教えてください。

1. 01. 作業フォルダとPowerShellを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
2. 02. Git・改行コード・秘密の除外を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
3. 03. VS Codeとrust-analyzerを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
4. 04. rustup・rustc・Cargoを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
5. 05. MSVC Build ToolsとCPUアーキテクチャを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
6. 06. Cargo品質コマンドを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
7. 07. WSL2とDockerを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
8. 08. Topcoat 0.5.0とCLIを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
9. 09. Turso CLIとlibSQL疎通を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
10. 10. flyctlと環境診断を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。

答える順序 まず資料を閉じて説明し、次に最小コードを書き、最後に該当単元へ戻って不足を補います。正解の暗記ではなく、根拠を再現できることが目標です。

REVIEW ANSWERS

確認問題・解答の骨格

次の要点は模範解答の丸暗記用ではありません。自分の説明に「定義・小さな例・バックエンドでの場所」が含まれているかを照合します。

01. 作業フォルダとPowerShell

作業フォルダとPowerShellとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 環境構築の診断は、シェル、現在位置、OS、CPUから始めます。

02. Git・改行コード・秘密の除外

Git・改行コード・秘密の除外とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 生成物と秘密は除外し、依存固定とDB移行は履歴へ入れます。

03. VS Codeとrust-analyzer

VS Codeとrust-analyzerとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: エディタ診断の対象ルートと、Cargoを実行するルートを一致させます。

04. rustup・rustc・Cargo

rustup・rustc・Cargoとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: rustupは工具箱、rustcは翻訳機、Cargoはプロジェクト監督です。

05. MSVC Build ToolsとCPUアーキテクチャ

MSVC Build ToolsとCPUアーキテクチャとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: リンカエラーではRustコードより先にhost triple、Build Tools、CPU、ネイティブ依存を確認します。

06. Cargo品質コマンド

Cargo品質コマンドとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: check・fmt・clippy・testは役割が違い、全部通って初めて変更を統合します。

07. WSL2とDocker

WSL2とDockerとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: WSLは開発環境、Dockerは配布環境の再現に使い、ツールを入れた場所を区別します。

08. Topcoat 0.5.0とCLI

Topcoat 0.5.0とCLIとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 変化の速いTopcoatはcrate、CLI、Cargo.lock、参照APIを同じ版へ固定します。

09. Turso CLIとlibSQL疎通

Turso CLIとlibSQL疎通とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: DB URLは場所、tokenは秘密です。CLI出力を共有するときはtokenを伏せます。

10. flyctlと環境診断

flyctlと環境診断とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: Fly.ioではアプリ、組織、リージョン、待受、秘密、ヘルスチェックを別々に確認します。

GLOSSARY

重要用語と実務での位置

用語	一文定義	バックエンドでの位置
作業フォルダとPowerShell	環境構築の診断は、シェル、現在位置、OS、CPUから始めます。	開発、テスト、Dockerで同じプロジェクトルートを使い、手順には現在位置を明記して再現性を上げます。
Git・改行コード・秘密の除外	生成物と秘密は除外し、依存固定とDB移行は履歴へ入れます。	Cargo.lock、migrations、fly.tomlは履歴へ入れ、target、.env、秘密鍵は除外します。アプリはCargo.lockを固定して再現性を優先します。
VS Codeとrust-analyzer	エディタ診断の対象ルートと、Cargoを実行するルートを一致させます。	エディタ診断は補助です。commit前とCIではプロジェクトルートから同じCargoコマンドを実行します。
rustup・rustc・Cargo	rustupは道具箱、rustcは翻訳機、Cargoはプロジェクト監督です。	CIとDockerでもrust-versionとeditionを合わせます。rustup updateだけでは依存クレートの変化を固定できないためCargo.lockも必要です。
MSVC Build ToolsとCPUアーキテクチャ	リンカエラーではRustコードより先にhost triple、Build Tools、CPU、ネイティブ依存を確認します。	libSQLのようにネイティブコードを含む依存はリンカ、CMake、bindgenの影響を受けます。Windowsで詰まる場合はWSL2またはDockerで切り分けます。
Cargo品質コマンド	check・fmt・clippy・testは役割が違い、全部通って初めて変更を統合します。	バックエンドではbinだけでなくtests、examples、feature組合せも壊れます。CIでは警告をエラー扱いし、ローカルと同じコマンドを使います。
WSL2とDocker	WSLは開発環境、Dockerは配布環境の再現に使い、ツールを入れた場所を区別します。	Fly.ioはLinuxコンテナで動くため、最終ビルドをDockerで再現できるとWindows固有のリンカ差を本番へ持ち込みにくなります。
Topcoat 0.5.0とCLI	変化の速いTopcoatはcrate、CLI、Cargo.lock、参照APIを同じ版へ固定します。	0.5.0のAssetBundleは開発時にtopcoat dev、release時にcargo build --release後のtopcoat asset bundle --releaseが必要です。
Turso CLIとlibSQL疎通	DB URLは場所、tokenは秘密です。CLI出力を共有するときはtokenを伏せます。	URLは接続先、tokenは認証情報です。tokenを教材の画面やGitへ貼らず、開発では.env、本番ではfly secretsへ保存します。
flyctlと環境診断	Fly.ioではアプリ、組織、リージョン、待受、秘密、ヘルスチェックを別々に確認します。	fly launchでfly.tomlを作り、HOST=0.0.0.0、internal_port=8080、health checkを一致させます。秘密はfly secrets setで注入します。

PRIMARY SOURCES

公式資料と更新確認

仕様は更新されます。教材で意図を理解した後、実装時点の一次資料でAPI、security note、breaking changeを確認してください。

1. Rust Book

<https://doc.rust-lang.org/stable/book/>
Rust 2024 Edition を前提にした公式入門書

2. Rust Reference

<https://doc.rust-lang.org/reference/>
言語仕様を確認する一次資料

3. Standard Library

<https://doc.rust-lang.org/std/>
標準ライブラリAPI

4. Cargo Book

<https://doc.rust-lang.org/cargo/>
ビルド、依存関係、workspace、features

5. Rustdoc Book

<https://doc.rust-lang.org/rustdoc/>
APIドキュメントの書き方

6. Clippy

<https://doc.rust-lang.org/clippy/>
Rust公式Lint

7. Async Book

<https://rust-lang.github.io/async-book/>
非同期Rustの公式解説

8. Tokio Tutorial

<https://tokio.rs/tokio/tutorial>
Tokio公式チュートリアル

9. Topcoat

<https://github.com/tokio-rs/topcoat>
Topcoat公式リポジトリ。early-stageの注意を含む

10. Topcoat 0.5.0 API

<https://docs.rs/topcoat/0.5.0/topcoat/>
本教材で固定するAPI

11. Turso Rust SDK

<https://docs.turso.tech/sdk/rust/quickstart>
libSQL Rust SDK公式手順

12. Better Auth Database

<https://better-auth.com/docs/concepts/database>
Better AuthのDB構成

13. Fly.io Deploy

<https://fly.io/docs/launch/deploy/>
fly launch / fly deploy公式手順

14. Fly.io Health Checks

<https://fly.io/docs/reference/health-checks/>
公開後のヘルスチェック

15. OWASP Cheat Sheets

<https://cheatsheetseries.owasp.org/>
Webセキュリティの実務チェック

採用判断 本教材はRust 2024 Edition、Topcoat 0.5.0固定、Turso/libSQL、Topcoat Session + Argon2id、Fly.ioを主経路にします。Better AuthはTypeScript sidecarが適切な要件で比較します。